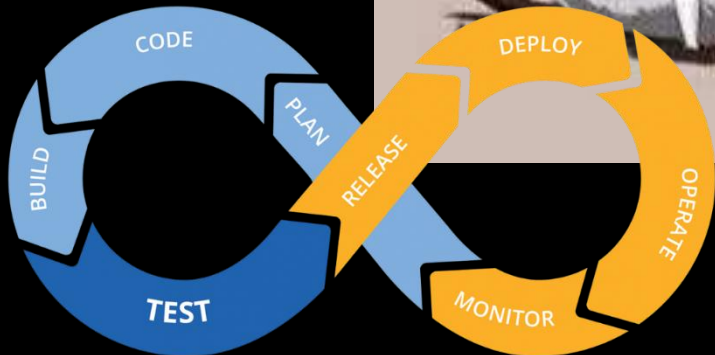


C Programming

17_testing

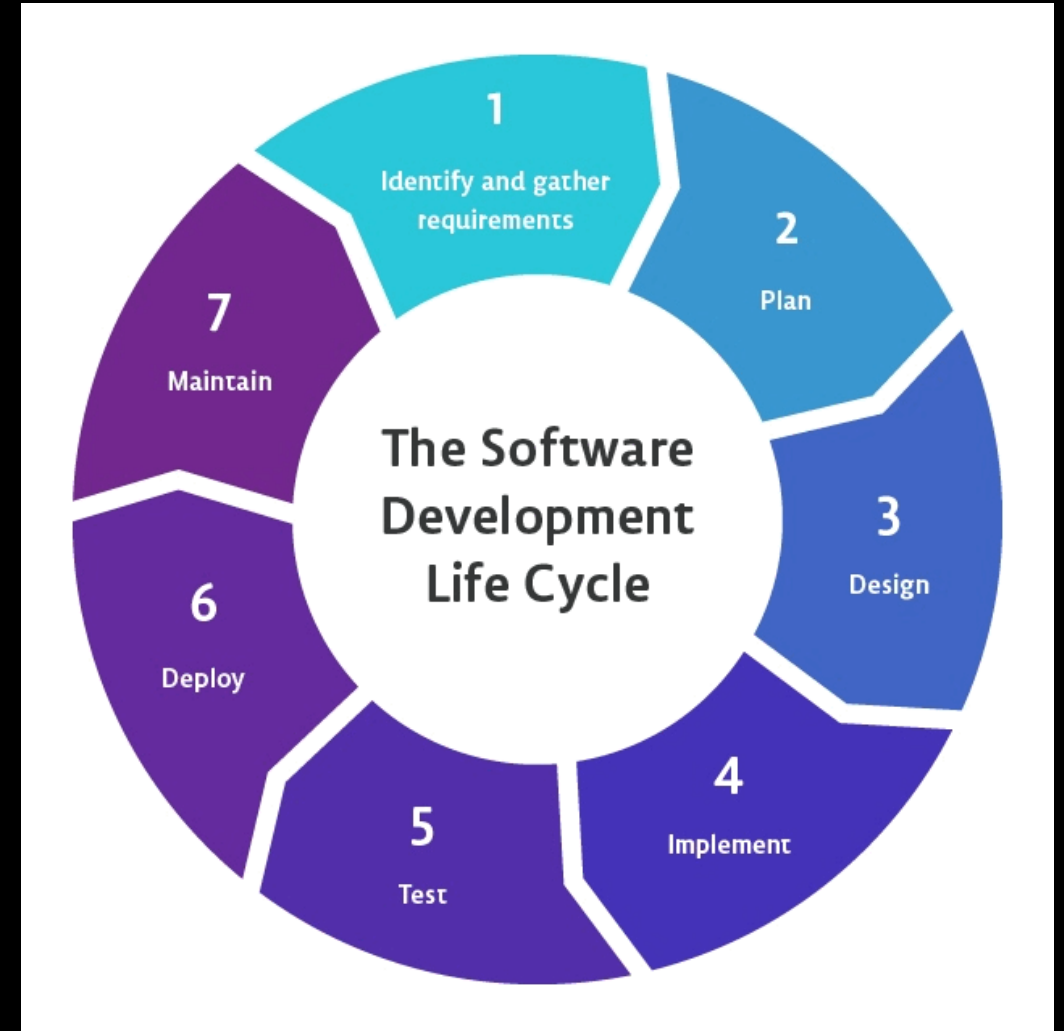
Тестирование ПО



Жизненный цикл разработки ПО (SDLC)

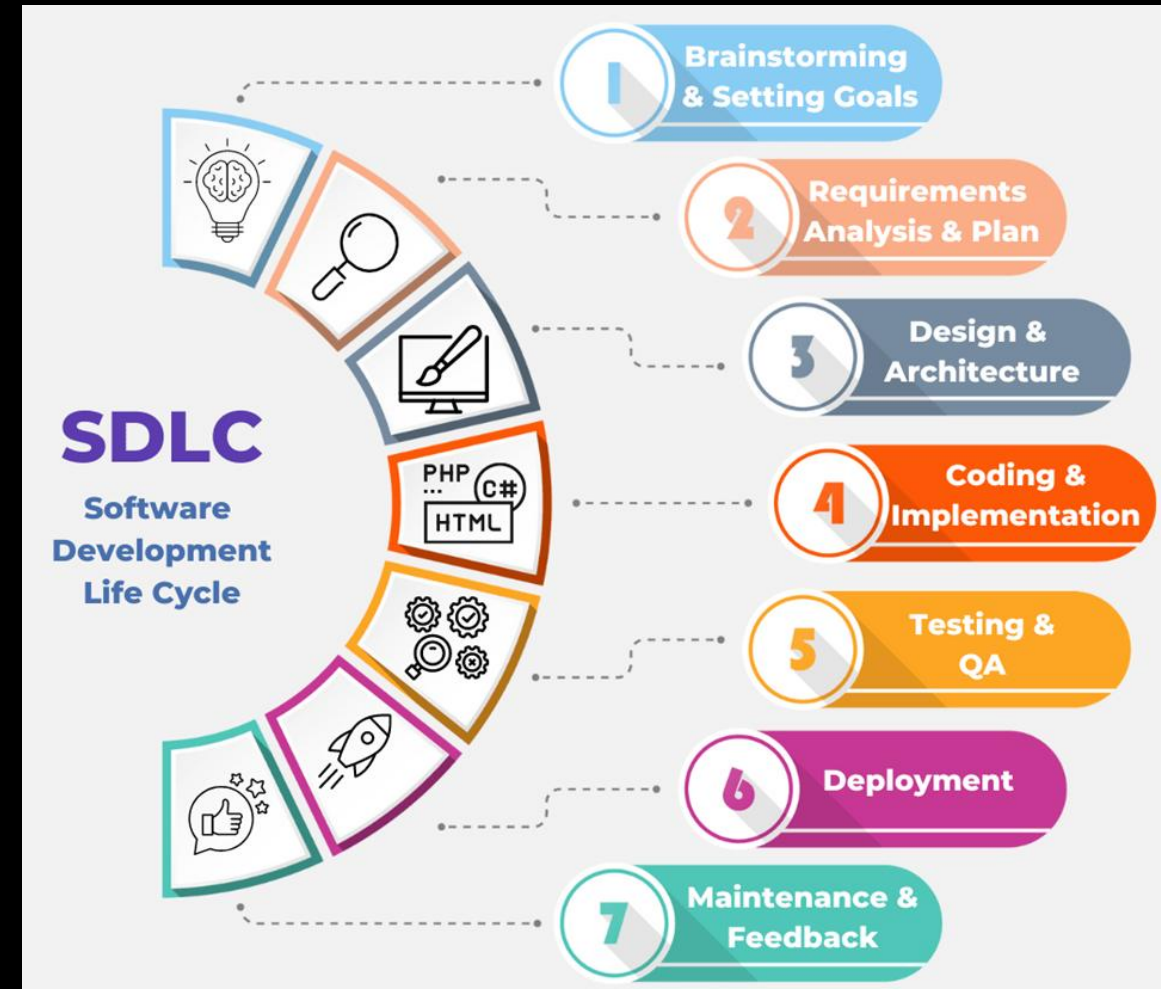
SDLC (Software Development Life Cycle) — это процесс, которому следуют для создания программного обеспечения в организациях. SDLC состоит из некоторого плана, который описывает, как разрабатывать, поддерживать, заменять и улучшать программное обеспечение.

На каждом этапе обычно есть свои должности: разработчик, тестировщик (QA-инженер), DevOps инженер, бизнес-аналитик, руководитель проекта, архитектор, тех. пис и т.д.



Жизненный цикл разработки ПО (SDLC)

1. Анализ требований
2. Планирование
3. Дизайн/Проектирование архитектуры
 - * HLD (High Level Design)
 - * LLD (Low Level Design)
4. Разработка продукта
5. Тестирование и интеграция продукта
6. Развертывание и обслуживание продуктов



Тестирование ПО

Тестирование ПО — это процесс проверки и подтверждения того, работает ли приложение так, как ожидалось, включает в себя выявление ошибок и неполадок, которые в будущем могут вызвать проблемы у пользователей приложения.

Ручное тестирование (manual) - тестирование приложения вручную. Тестировщик напрямую взаимодействует с приложением и смотрит работает ли как надо. Если работает не как надо - есть баги, то заводит задачи на разработчиков. В любом случае дает фидбек (обратную связь)

Автоматизированное тестирование (Automation) - значит написать код, в котором ожидаемые сценарии сравниваются с тем, что действительно получил пользователь. Простыми словами - тестировщик пишет код/скрипты, которые тестируют программу.

Виды/типы тестов

- **Модульные тесты (Unit-тесты).** Модульные тесты работают на очень низком уровне. Они заключаются в тестировании отдельных функций, компонентов или модулей, используемых в ПО. Модульные тесты обычно легковесны и выполняются быстро
- **Интеграционные тесты (Integration).** В ходе интеграционного тестирования проверяется, хорошо ли работают вместе различные модули, используемые приложением. Например, можно протестировать взаимодействие с базой данных.
- **Функциональные тесты (Functional).** В функциональных тестах основное внимание уделяется бизнес-требованиям к приложению. Они проверяют только результат некоторого действия и не проверяют промежуточные состояния системы при выполнении этого действия.

Иногда возникает путаница между понятиями интеграционных и функциональных тестов, так как и те и другие требуют взаимодействия нескольких компонентов друг с другом. Разница в том, что интеграционный тест нужен просто чтобы убедиться, что вы можете отправлять запросы к базе данных, тогда как функциональный тест будет ожидать получения из базы данных определенного значения в соответствии с требованиями продукта.

<https://www.atlassian.com/ru/continuous-delivery/software-testing/types-of-software-testing>

Виды/типы тестов

- **Сквозные тесты (End to End или E2E).** Сквозное тестирование копирует поведение пользователя при работе с ПО в контексте всего приложения. Оно обеспечивает контроль того, что различные схемы действий пользователя работают должным образом. Сценарии могут быть как очень простыми (загрузка веб-страницы или вход в систему), так и гораздо более сложными (проверка почтовых уведомлений, онлайн-платежей и т. д.).
- **Приемочное тестирование (Acceptance).** Приемочные тесты — это формальные тесты, которые проверяют, отвечает ли система требованиям бизнеса. При этом во время тестирования должно быть запущено само приложение, и основное внимание уделяется воспроизведению поведения пользователей.
- **Тестирование производительности (performance).** В тестах производительности оценивается работа системы при определенной рабочей нагрузке.
- **Smoke-тестирование.** Smoke-тесты — это базовые тесты, которые проверяют основные функциональные возможности приложения. Они должны выполняться быстро, поскольку цель таких тестов — убедиться, что основные возможности системы работают как запланировано.

<https://www.atlassian.com/ru/continuous-delivery/software-testing/types-of-software-testing>

Фазы тестирования

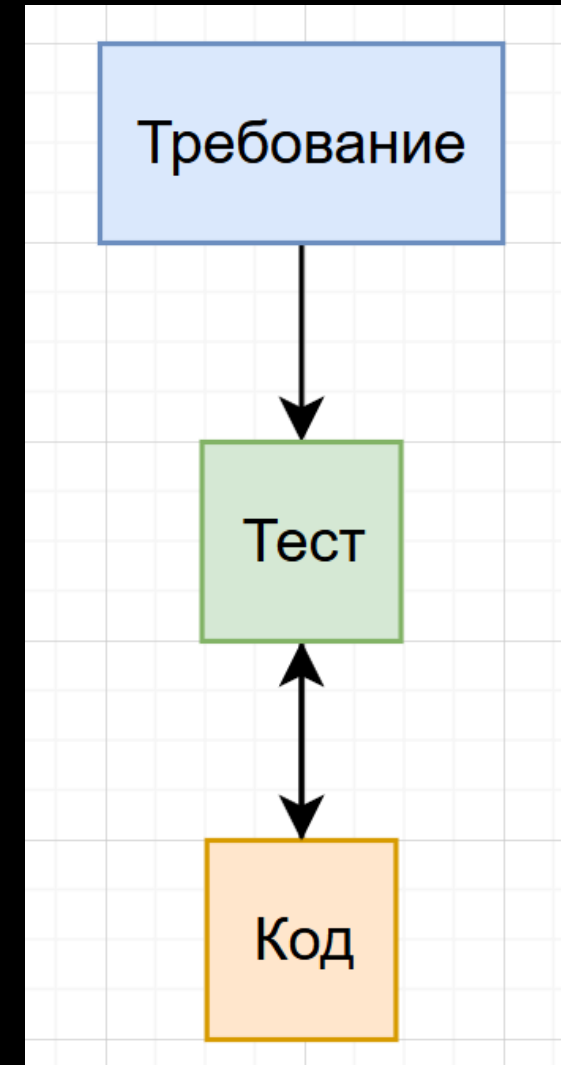
- **Pre-Alpha:** прототип, в котором всё ещё присутствует много ошибок и наверняка неполный функционал. Необходим для ознакомления с будущими возможностями программ.
- **Alpha:** является ранней версией программного продукта, тестирование которой проводится внутри фирмы-разработчика.
- **Beta:** практически готовый продукт, который разработан в первую очередь для тестирования конечными пользователями.
- **Release Candidate (RC):** возможные ошибки в каждой из фичей уже устранены и разработчики выпускают версию на которой проводится регрессионное тестирование.
- **Release:** финальная версия программы, которая готова к использованию.

Behavior-driven development (BDD)

Для повышения качества кода программисты довольно давно придумали немного иной метод разработки, что обозвали BDD - Behavior-driven development, дословно «разработка через поведение».

Почему мы сначала пишем тесты, а для них код?

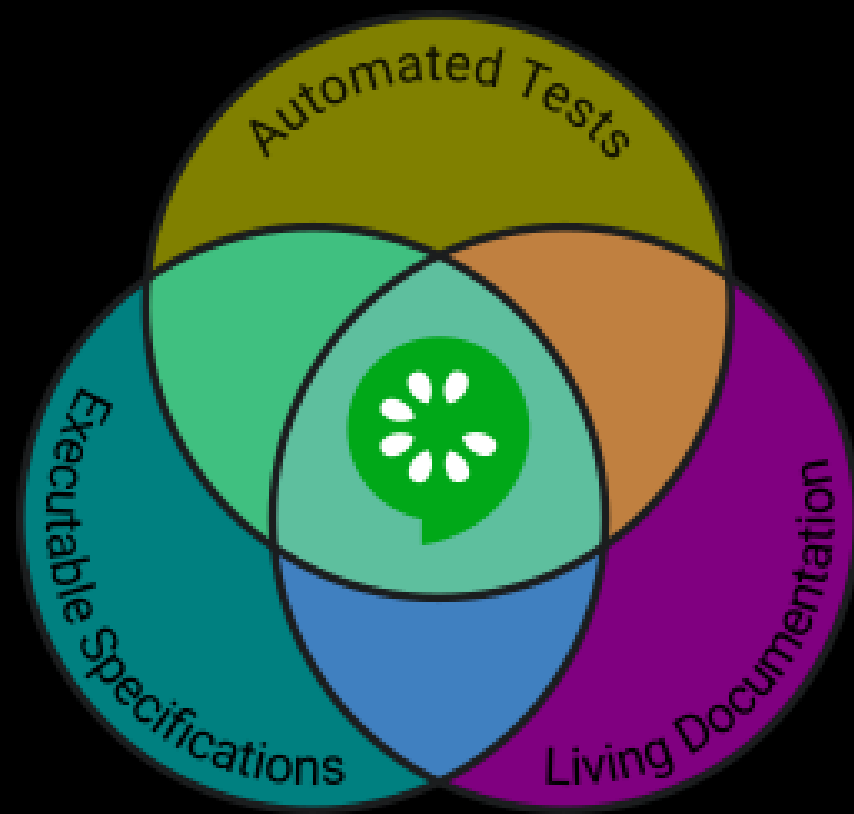
- Дело в том, что такая практика позволяет сфокусироваться на какой-то мелкой задаче и писать тот код, что решает только ее и не задумываться о том, что мой код должен решать сразу все мировые проблемы. Мой код должен решать этот тест и на этом все.
- Написание требований и тестов покрывает также документацию кода, которую разработчики так не любят писать



Gherkin

Given User is on Login Page
When User enters login 'user_login'
And User enters password 'user_password'
Then 'Welcome to our site' message displayed

Допустим пользователь находится на странице логина
Когда пользователь вводит логин 'user_login'
И пользователь вводит пароль 'user_password'
Тогда появляется сообщение 'Welcome to our site'

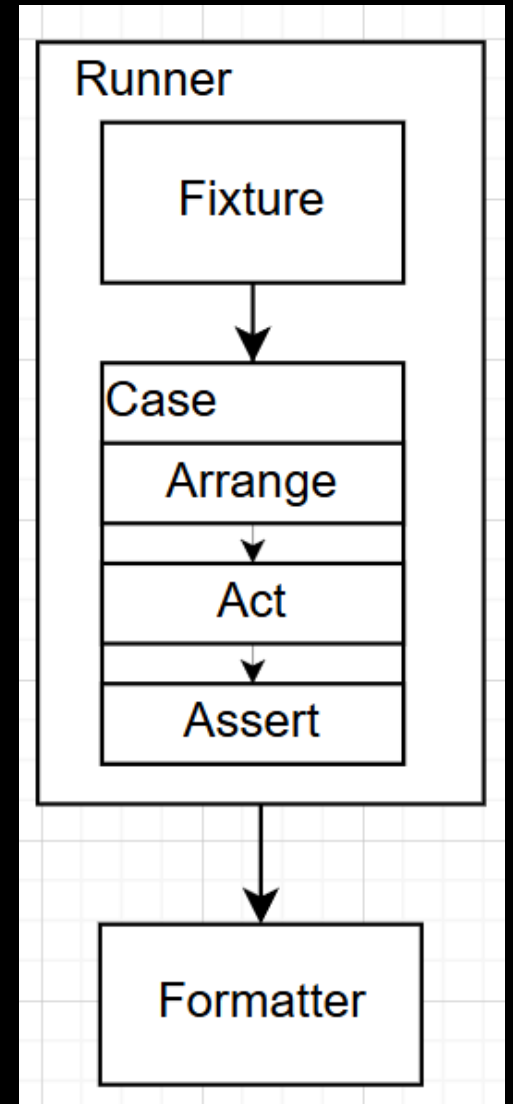


Тесты – это программа

Тесты - есть программа, чаще всего написанная на том же языке, на котором написан целевой код (Программы на Си обычно покрывают кодом на Си и т.д.)

Все тесты довольно похожи друг на друга, поэтому была придумана типовая архитектура фреймворка для тестирования – xUnit. Упрощенная схема того как обычно выглядит тестовая программа:

- Фикстура - некоторая конфигурация (настройки) тестирования. Например настройки в каком состоянии должна находиться система для тестирования
- Arrange - подготовить входные данные
- Act - Отработка теста
- Assert - Проверка
- Formatter - форматирование в красивом для человека вида с результатами тестов
- Runner - тот, кто запускает программу



AAA

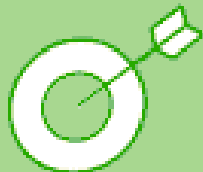
Arrange-Act-Assert (AAA)

ARRANGE



Setup the code to test

ACT



Perform the action you want to test

ASSERT



Check if the result matches your expectation

Фреймворки для тестирования

Список фреймворков для Unit тестов:

https://en.wikipedia.org/wiki/List_of_unit_testing_frameworks

Для языка Си как и для других языков есть очень много разных фреймворков для тестирования, но мы рассмотрим пару примеров на элегантном CMocka. Ссылки на доки:

<https://cmocka.org/>

https://cmocka.org/talks/cmocka_unit_testing_and_mocking.pdf

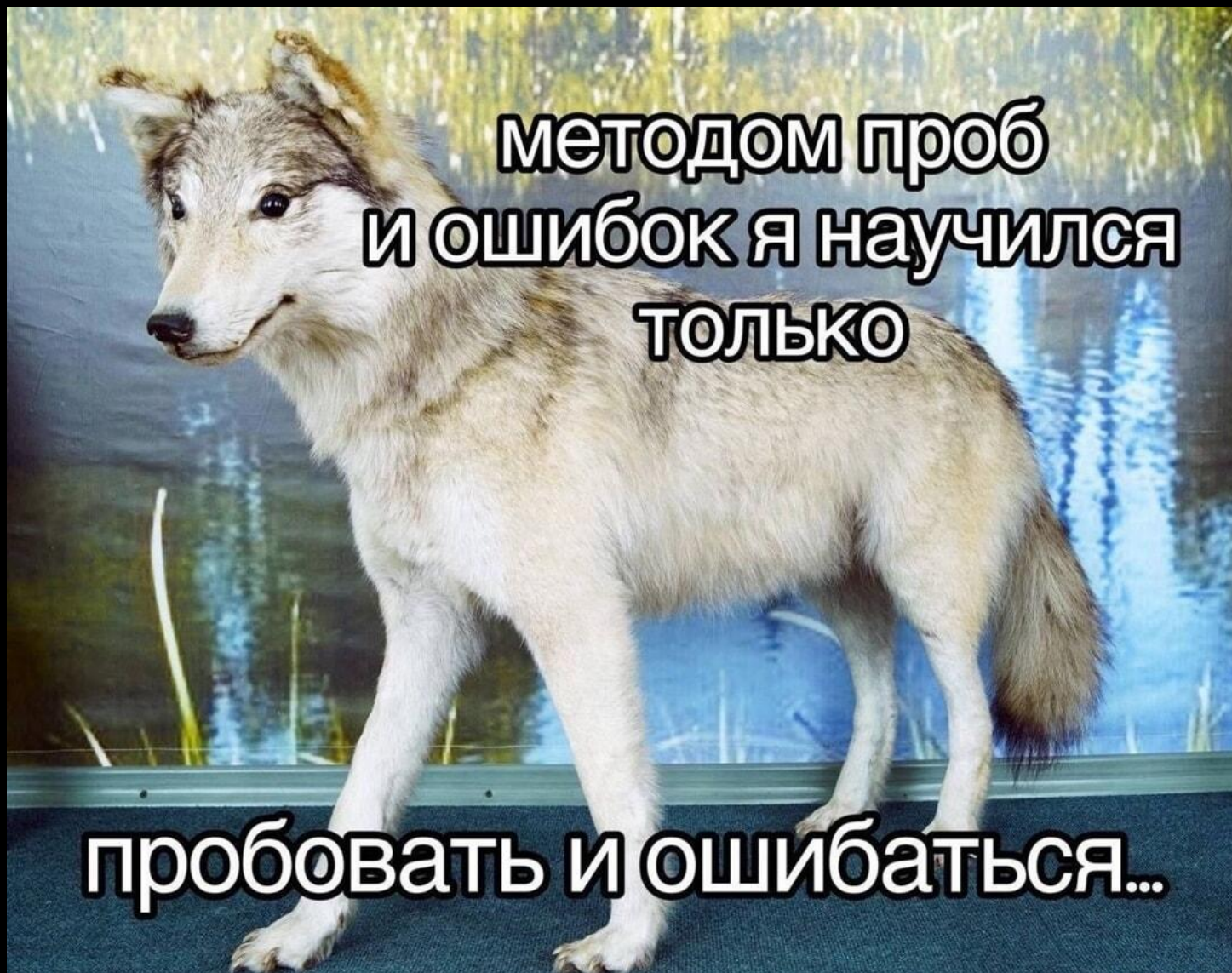
<https://api.cmocka.org/index.html>

Установить Cmocka можно, например, так:

```
sudo apt install libcmocka-dev
```

При компиляции линкуем с библиотекой флагом -lcmocka

Ура, программировать



Пару примеров:

https://github.com/kruffka/C-Programming/blob/2025-2026/17_testing/src/test_example.c

https://github.com/kruffka/C-Programming/blob/2025-2026/17_testing/src/test_file.c

Форматтер

Сmocka:

CMOCKA_MESSAGE_OUTPUT=stdout ./test

CMOCKA_MESSAGE_OUTPUT=xml ./test

Ctest и Cmake:

```
Test project /home/stephan/Documents/cpp_testing_sample/build
  Start 1: RegularUsage
1/8 Test #1: RegularUsage ..... Passed    0.00 sec
  Start 2: NoArg
2/8 Test #2: NoArg ..... Passed    0.00 sec
  Start 3: WrongArg
3/8 Test #3: WrongArg ..... Passed    0.00 sec
  Start 4: NegativeNo
4/8 Test #4: NegativeNo ..... Passed    0.00 sec
  Start 5: GeneratorTest.ReferenceCheck
5/8 Test #5: GeneratorTest.ReferenceCheck ..... Passed    0.00 sec
  Start 6: GeneratorTest.NumberOfElements
6/8 Test #6: GeneratorTest.NumberOfElements ... Passed    0.00 sec
  Start 7: GeneratorTest.Shuffle
7/8 Test #7: GeneratorTest.Shuffle ..... Passed    0.00 sec
  Start 8: GeneratorTest.CheckSum
8/8 Test #8: GeneratorTest.CheckSum ..... Passed    0.00 sec

100% tests passed, 0 tests failed out of 8

Total Test time (real) =  0.04 sec
```


Code Coverage (Покрытие кода)

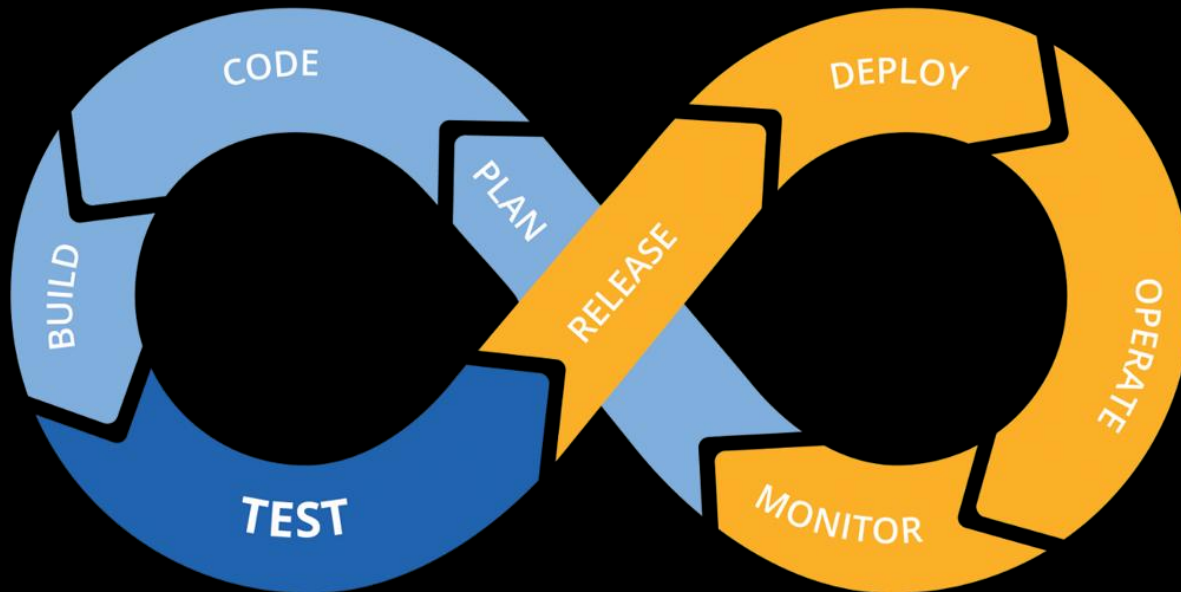
Code Coverage — это метрика покрытия кода, не метрика закрытия требований
Покрытие тестами на 100% != Качество кода

/									
98.36% Statements 1859/1890 93.92% Branches 726/773 100% Functions 266/266 99.73% Lines 1816/1821 37 statements, 21 branches Ignored									
File	Statements	Branches	Functions	Lines					
express/	100%	1/1	100%	0/0	100%	0/0	100%	1/1	
express/examples/auth/	93.75%	75/80	80.77%	21/26	100%	17/17	100%	71/71	
express/examples/content-negotiation/	100%	32/32	100%	2/2	100%	13/13	100%	32/32	
express/examples/cookie-sessions/	100%	12/12	100%	4/4	100%	1/1	100%	12/12	
express/examples/cookies/	95.83%	23/24	87.5%	7/8	100%	3/3	100%	22/22	
express/examples/downloads/	94.12%	16/17	87.5%	7/8	100%	3/3	100%	15/15	
express/examples/ejs/	100%	11/11	100%	2/2	100%	1/1	100%	11/11	
express/examples/error-pages/	97.14%	34/35	83.33%	10/12	100%	6/6	100%	34/34	
express/examples/error/	90%	18/20	66.67%	4/6	100%	4/4	100%	18/18	
express/examples/markdown/	95.24%	20/21	66.67%	4/6	100%	5/5	100%	20/20	
express/examples/multi-router/	100%	9/9	100%	2/2	100%	1/1	100%	9/9	
express/examples/multi-router/controllers/	100%	14/14	100%	0/0	100%	4/4	100%	14/14	
express/examples/mvc/	95.45%	42/44	80%	8/10	100%	4/4	100%	42/42	
express/examples/mvc/controllers/main/	100%	2/2	100%	0/0	100%	1/1	100%	2/2	
express/examples/mvc/controllers/pet/	94.12%	16/17	50%	1/2	100%	4/4	100%	16/16	
express/examples/mvc/controllers/user-pet/	92.86%	13/14	50%	1/2	100%	1/1	100%	13/13	
express/examples/mvc/controllers/user/	100%	21/21	100%	4/4	100%	6/6	100%	19/19	
express/examples/mvc/lib/	97.96%	48/49	80%	20/25	100%	2/2	100%	46/46	

CI/CD

Непрерывная интеграция (Continuous Integration, CI) и **непрерывная поставка (Continuous Delivery, CD)** представляют собой культуру, набор принципов и практик, которые позволяют разработчикам чаще и надежнее разворачивать изменения программного обеспечения.

Непрерывная интеграция — это методология разработки и набор практик, при которых в код вносятся небольшие изменения с частыми коммитами. И поскольку большинство современных приложений разрабатываются с использованием различных платформ и инструментов, то появляется необходимость в механизме интеграции и тестировании вносимых изменений



<https://habr.com/ru/companies/otus/articles/515078/>

<https://github.com/kruffka/C-Programming>

